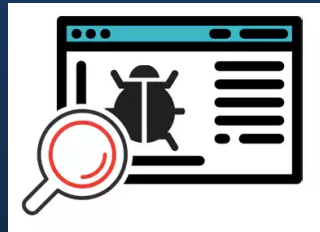


Zasady tworzenia testowalnego kodu



„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Grzegorz Szymkowiak

Jak tworzyć testowalny kod?

Tworzenie testowalnego kodu

- Kod testowalny to taki, który łatwo:
 - izolować
 - uruchamiać automatycznie
 - nie zależy od środowiska
 - nie wymaga ręcznej konfiguracji
 - nie zależy od infrastruktury

Jedna odpowiedzialność

```
double add(double a, double b) {  
    return a + b;  
}  
  
double sub(double a, double b) {  
    return a - b;  
}  
  
double mult(double a, double b) {  
    return a * b;  
}
```

Dany element kodu powinien odpowiadać za jeden obszar odpowiedzialności biznesowej

Funkcja deterministyczna

- Funkcja deterministyczna to taka funkcja, która dla tych samych danych wejściowych zawsze zwraca ten sam wynik, a także:
 - nie zależy od środowiska
 - nie korzysta z ukrytego stanu
 - nie zależy od czasu

Czysta funkcja

- Czysta funkcja to taka, która:
 - dla takich samych danych wejściowych daje zawsze taki sam wynik
 - nie zależy od danych z zewnątrz
 - nie ma skutków ubocznych, np. nie loguje do konsoli

```
double add(double a, double b) {  
    return a + b;  
}
```

Czysta funkcja

- Czysta funkcja jest idealna dla testów jednostkowych, ponieważ:
 - łatwo ją testować (tylko wejście i wyjście)
 - nie zależy od środowiska
 - nie psuje niczego poza zakresem
 - nie zależy od kolejności wywołań

Izolacja

- Kod jest izolowany, gdy można go uruchamiać i testować niezależnie od innych elementów systemu, tzn.:
 - komponent działa samodzielnie
 - jego zachowanie nie zależy od struktury
 - można go sprawdzić bez uruchomienia całej aplikacji

Izolacja

- Izolację uzyskuje się poprzez:
 - usuwanie twardej zależności
 - wprowadzenie abstrakcji
 - rozdzielenie logiki od infrastruktury

Izolacja

```
// Interfejs repozytorium użytkowników
class IUserRepository
{
public:
    virtual bool SaveUser(const std::string& name) = 0;

    virtual ~IUserRepository() = default;
};

// Implementacja repozytorium użytkowników, która zapisuje dane do bazy danych
class DatabaseRepository : public IUserRepository
{
public:
    bool SaveUser(const std::string& name) override
    {
        // prawdziwa baza danych
        return true;
    }
};
```

Izolacja

```
// Serwis użytkowników, który korzysta z repozytorium do rejestracji użytkowników
class UserService
{
    IUserRepository& repository;

public:
    UserService(IUserRepository& repo) : repository(repo) {}

    bool Register(const std::string& name)
    {
        if (name.empty()) return false;

        return repository.SaveUser(name);
    }
};
```

www.edunet.tychy.pl

KONIEC

Grzegorz Szymkowiak